[Week 10]

chelsseey · 2025년 4월 29일

Euron

통계 수정 삭제

0

05. 토큰화

자연어(Natural Language)
: 사람들 사이의 의사소통을 위해 자연스럽게 형성된 언어

자연어 처리(NLP, Natural Language Processing)
: 컴퓨터가 인간의 언어를 이해·해석·생성할 수 있도록 하는 인공지능 기술

NLP 모델 개발에서 해결해야 할 핵심 문제

- 모호성
  같은 단어·구절이 문맥에 따라 여러 의미로 해석될 수 있음.
  알고리즘은 다양한 의미를 구별해야 함.
- 가변성
  사투리, 강세, 신조어, 작문 스타일 등의 차이로 표현이 매우 다양함.
  다양한 언어 변형을 처리해야 함.
- 구조
  구문(syntax)을 파악해 의미(semantics)를 정확히 해석해야 함.
  문법적 요소를 기반으로 의미를 추론해야 함.

말뭉치(Corpus)
: 뉴스 기사·사용자 리뷰·저널·칼럼 등 목적에 따라 수집·구축한 대규모 텍스트 데이터

토큰(Token)과 토큰화(Tokenization)

- 토큰(Token)
  말뭉치보다 작은 단위: 개별 단어·구두점·의미 단위
- 토큰화(Tokenization)
  텍스트를 토큰 단위로 쪼개는 전처리 과정
  컴퓨터가 언어의 구조와 의미를 효과적으로 분석·처리할 수 있도록 함

```
입력: 형태소 분석기를 이용해 간단하게 토큰화할 수 있다.
결과: ["형태소", "분석기", "를", "이용", "해", "간단하게", "토큰화", "할", "수", "있다", "."]
```

→ "…"로 둘러싸인 각 문자열 `"형태소"`, `"분석기"`, `"를"` 등이 하나의 토큰

토크나이저(Tokenizer)
: 문자열을 토큰으로 분할하는 알고리즘 또는 소프트웨어

토크나이저 구축 방법

- 공백 분할
  스페이스(공백) 기준으로 단어 분리
- 정규표현식 적용
  특정 패턴(숫자, 이메일, URL 등)을 인식해 분리
- 어휘 사전 기반
  사전에 정의된 단어집합을 활용
  OOV 문제: 사전에 없는 단어(Out-Of-Vocab) 처리 필요
- 머신러닝 활용
  말뭉치 데이터를 기반으로 토큰 경계를 학습

대규모 어휘 사전의 한계

- 학습 비용 증가
- 차원의 저주 (Curse of Dimensionality)
  벡터 차원이 어휘 크기와 동일 → 고차원 희소 벡터

```
ex. 어휘 사전(Vocabulary)에 10,000개의 서로 다른 단어(토큰)
"사과" → [0, 0, …, 1, …, 0]   (10,000차원 중 하나만 1, 나머지는 0)
```

- 희소(Sparse) 표현의 문제
  벡터 대부분이 0이고 극히 일부만 1인 벡터 = 희소(sparse) 벡터
  출현 빈도만 반영 → 드문 토큰 정보 학습 어려움
- 순서 정보 미반영
  토큰 순서(order)나 문맥(context) 무시

단어 및 글자 토큰화

토큰화(Tokenization)
: 텍스트를 가장 작은 의미 단위인 토큰으로 분해하는 전처리 과정
단어·문장 부호·글자 등의 빈도·출현 패턴을 파악 → 분석·학습의 기초 자료로 활용

단어 토큰화

띄어쓰기·문장 부호 등을 기준으로 텍스트를 개별 단어 단위로 분리

split() 기반 단어 토큰화
: 문자열에서 지정된 구분자(기본값: 공백) 기준으로 잘라서 토큰 리스트를 생성

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = review.split()
print(tokenized)
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!
```

- OOV(사전에 없는 토큰)
  단어 토큰화로 만든 단어 사전에서 유사 토큰 `cg` , `cg.` 을 별개로 취급
- 한국어 접사, 문장 부호, 오타, 띄어쓰기 오류에 취약

글자 토큰화

텍스트를 단어가 아니라 글자(문자) 단위로 분해하는 토큰화 기법

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = list(review)
print(tokenized)
['현','실','과',' ','구','분',' ', … ,'!','!']
```

공백, 구두점, 한글 글자, 영문자까지 모두 한 글자씩 분리되어 리스트로 반환됨

- 작은 단어 사전만으로도 구현 가능 → OOV 문제 완화

한글 자소 단위 토큰화

글자 토큰화보다 더 작은 단위인 자소(초성·중성·종성)까지 분해
어휘 크기를 더욱 줄여 학습 효율↑, OOV↓

조합형 vs 완성형 한글 인코딩

- 조합형(Jamo-based): 초성·중성·종성을 개별 자모 코드로 저장
  → 문자를 조합해 한 글자를 표현
- 완성형(Precomposed): 미리 조합된 글자(ex. 가, 나, 다)에
  고유 코드값을 부여해 저장

`h2j` 함수
: 완성형 한글 문자열 → 조합형 자모 시퀀스로 변환
모든 한글 글자를 "초성·중성·종성" 단위로 분해할 준비를 함

`j2hcj` 함수
: 조합형 자모 → 자소(호환 자모) 분해
초성·중성·종성이 개별 문자로 분리된 시퀀스를 반환

```
from jamo import h2j, j2hcj

review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고!"
decomposed = j2hcj(h2j(review))
tokens     = list(decomposed)
print(tokens)
['ㅎ','ㅕ','ㄴ','ㅅ','ㅣ','ㄹ','ㄱ','ㅘ',' ','ㄱ','ㅜ','ㅂ','ㅜ','ㄴ',' ','ㅂ','ㅜ','ㄹ','ㄱ','ㅏ','ㄴ',' ','c','g','.',' ', …
```

- 개별 토큰이 의미를 가지지 않고 조합하여 결과를 도출해야 함
- 다의어·동음이의어 처리가 어려움
- 입력 시퀀스 길이 증가에 따른 계산 비용 부담

형태소 토큰화

: 텍스트를 의미의 최소 단위인 형태소(morpheme)로 분해하는 전처리 기법
어근·접사·조사·어미 등을 분리해 "의미 단위"로 나누고,
문법·구조적 정보를 반영한 토큰화

한국어에서의 필요성

한국어 어절 하나에 어근(root)과 여러 형태소(조사·어미·접사)가 결합
→ 교착어 특성 : 형태소 단위로 분리해서 처리해야함

형태소의 종류

형태소 종류	설명	예시
자립 형태소	스스로 의미를 갖는 기본 단위 (Free Morpheme)	명사(나, 책), 동사(가다), 형용사(크다)
의존 형태소	단독 사용 불가, 자립형태소와 결합해 역할 수행 (Bound Morpheme)	조사(은, 에게), 어미(했다), 접두·접미사(미-,-하다)

```
입력 문장 : 그는 나에게 인사를 했다.
형태소 토큰화 결과 : ['그', '는', '나', '에게', '인사', '를', '하', '었', '다', '.']
```

형태소 어휘 사전(Morpheme Vocabulary)

형태소 단위로 구성된 어휘 집합으로, 각 형태소의 의미·품사 정보를 포함

- "그는/그녀는/나는"처럼 유사 어근에 다른 조사·어미가 결합한 형태를 일관되게 처리
- OOV(사전 미등장) 형태소를 최소화 → 학습 효율↑

품사 태깅(POS Tagging)

형태소 분석 결과에 각 형태소의 품사 정보를 부착하는 작업
문맥·문법 역할 파악 → 개체명 인식, 구문 분석, 의미 해석 정교화

```
품사 태깅 예시 :
그 ( 명사 ) + 는 ( 조사 ) + 나 ( 명사 )+ 에게 ( 조사 )+ 인사 ( 명사 ) + 를 ( 조사 )+ 했다 ( 동사 )
```

KONLPy

국어 자연어 처리를 위해 개발된 자바(JDK) 기반의 라이브러리

- 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공
- Okt(Open Korean Text), 꼬꼬마 (Kkma), 코모란 (Komoran),
  한나눔 (Hannanum),메캅 (Mecab) 등의 다양한 분석기 지원

Okt(Open Korean Text)

SNS 텍스트 데이터를 기반으로 개발된 형태소 분석기

주요 메서드

메서드	설명	반환 형태
`okt.nouns()`	문장에서 **명사**만 추출	리스트 of 문자열
`okt.phrases()`	문장에서 **의미 단위의 구(phrase)** 추출	리스트 of 문자열
`okt.morphs()`	문장에서 **모든 형태소** 추출	리스트 of 문자열
`okt.pos()`	문장에서 **형태소+품사** 태깅	리스트 of (형태소, 품사) 튜플

Kkma

문장 단위 분할(sentences) 지원
비교적 상세한 형태소 분해와 56개 이상의 품사 태그 제공

Okt vs. Kkma 비교

항목	Okt	Kkma
문장 분리	지원하지 않음	`kkma.sentences()` 가능
명사 추출 예시	`['무엇','상상','수','사람','수']`	`['무엇','상상','수','사람','무엇']`
형태소 분해	합쳐진 어간/어미를 일부 미분리	더 세밀하게 어간·어미·접사 분리
품사 태그 수	19개	56개 이상
속도·자원	빠르고 가벼움	상대적으로 느리고 무거움

NLTK

영어를 비롯해 네덜란드어·프랑스어·독일어 등 다양한 언어의
토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 기능 제공

• 영어 문장·단어 토큰화

- word_tokenize: 문장을 단어 단위로 분리, 구두점·콤마 처리
- sent_tokenize: 텍스트를 ". ! ?" 등을 기준으로 문장 단위로 분리

```
from nltk.tokenize import word_tokenize, sent_tokenize

sentence = "Those who can imagine anything, can create the impossible."
word_tokens = word_tokenize(sentence)
sent_tokens = sent_tokenize(sentence)

# word_tokens → ['Those', 'who', 'can', 'imagine', 'anything', ',', 'can', 'create', 'the'
# sent_tokens → ['Those who can imagine anything, can create the impossible.']
```

• 품사 태깅
averaged_perceptron_tagger 모델 기반으로 각 단어에 POS 태그 부착

```
tagged = nltk.pos_tag(word_tokens)
# tagged → [('Those','DT'),('who','WP'),('can','MD'), … ('impossible','.')]
```

품사 태그 : DT(한정사), WP(의문대명사), MD(조동사), NN(명사) 등
→ Penn Treebank 기준

spaCy

Cython 기반의 고성능 NLP 라이브러리
NLTK보다 빠르고 정확도를 중시하며, GPU 가속 지원
영어·한국어·일본어 등 24개 이상 언어의 사전 학습 모델 제공

spaCy 처리 흐름

- 모델 로드

```
import spacy
nlp = spacy.load("en_core_web_sm")
```

- 문장 처리
  doc 객체에 토큰화·품사 태깅·의존구문 분석 등의 결과가 담김

```
doc = nlp("Those who can imagine anything, can create the impossible.")
```

- 토큰 및 품사 정보 추출
  `token.text` : 원본 텍스트
  `token.pos_` : Universal POS 태그(간소화된 품사)
  `token.tag_` : 상세 POS 태그(Penn Treebank 기준)

```
for token in doc:
    print(f"[{token.pos_:5} - {token.tag_:3}] : {token.text}")
```

- 출력

```
[PRON - DT ] : Those
[PRON - WP ] : who
[AUX  - MD ] : can
[VERB - VB ] : imagine
…
[PUNCT- .  ] : .
```

일관된 파이프라인(토크나이저→태거→파서)을 객체 지향(doc/token)으로 제공

하위 언어 토큰화

하나의 토큰(단어)을 더 작은 의미 단위인 하위 단어(subword) 조합으로 분할

```
Reinforcement → ['Rein', 'force', 'ment']
```

전통 형태소 토큰화의 한계

- 신조어·오탈자·외래어 등 어휘가 빠르게 변하며,
  형태소 분석기는 이들을 사전에 반영하기 어려움
- OOV(Out-Of-Vocab) 문제
  사전에 없는 단어가 계속 늘어나면 어휘집 크기가 급격히 커짐
- 분해 단위의 불일치
  '돈쭐내다'는 하나의 신조어지만, 형태소 분석기로 토큰화하면 ['돈쭐날','만','하네','요'] 등 엉뚱하게 분리됨

하위 언어(Subword) 토큰화의 장점

- 어휘집 크기 절감
  단어 전체를 사전에 추가할 필요 없이, 자주 쓰이는 subword만 관리
- OOV 완화
  신조어·오래된 어휘·오타도, 부분 단위로 분해해 처리 가능
- 시퀀스 길이 감소
  글자 단위나 자소 단위보다 연산 효율↑

대표적 알고리즘 : Byte Pair Encoding (BPE), WordPiece, Unigram

바이트 페어 인코딩 (BPE)

데이터 압축용 알고리즘인 다이그램 코딩(digram coding)에서 파생된
하위 언어 토큰화 기법

텍스트 데이터에서 가장 자주 등장하는 연속 문자 쌍(pair)을 찾아
하나의 새로운 토큰(subword)으로 병합하며,
병합 가능한 쌍이 더 이상 없거나 사전 크기 한도에 도달할 때까지 반복

• ex. 'abracadabra'

- 초기 어휘
  모든 문자 → `['a','b','r','a','c','a','d','a','b','r','a']`
- 반복 병합
  1회차: 가장 빈번한 쌍 'ab' → 새로운 토큰 `A` 로 치환
  2회차: 치환 후 가장 빈번한 쌍 'ra' → 토큰 `B`
  3회차: 빈번한 쌍 `'AB'` → 토큰 `C`
  더 이상 병합할 쌍이 없으면 종료 → 압축 결과 CcadC

```
abracadabra → A racadA ra
AracadAra → AB cadAB
ABcadAB → CcadC
```

- 사전 구축: 매 병합 시마다 새로운 subword
  (ex. `'ab'` , `'ra'` , `'AB'`)를 어휘 사전에 추가

말뭉치에 바이트 페어 인코딩 적용

- 말뭉치 빈도 사전

```
[('low',5),('lower',2),('newest',6),('widest',3)]
```

- 문자 단위로 초기 분해

```
[('l','o','w',5),('l','o','w','e','r',2), …]
```

- 반복 병합
  가장 자주 등장하는 문자 쌍(`'e','s'`) → `'es'` 토큰 추가
  다음으로 자주 등장하는 쌍(`'es','t'`) → `'est'` 추가
  총 10회 반복 후 어휘 사전에는
  `['d','e','i','n','o','r','s','t','w','es','est','lo','low','ne','new','newest','wi','wid','widest']` 등 subword들이 등록
- OOV 완화
  새 단어 `'newer','wider','lowest'` 입력 시
  OOV 없이 기존 어휘(`new` , `e r` , `wid` , `er` , `low` , `est`) 조합으로
  토큰화 가능

센텐스피스

SentencePiece
: Google이 개발한 언어 독립적 하위단어 토크나이저 라이브러리

Korpora
: AI Hub·국립국어원 등에서 공개된 한국어 말뭉치를 손쉽게 불러올 수 있는 파이썬 패키지

워드피스(WordPiece) 토크나이저

BPE와 유사한 반복 병합 과정을 수행하되,
빈도(frequency) 기반이 아니라 확률(probability) 기반으로
가장 설명력이 높은 subword를 선택

각 단계에서 새로 병합된 subword가 전체 모델 확률(로그우도)을
최대화하도록 어휘를 구성

글자 쌍 병합 점수 계산

두 subword x, y 를 병합할지 판단하는 score는

$$\mathrm{score}(x, y) = \frac{f(xy)}{f(x)f(y)}$$

$f(\cdot)$: 말뭉치에서 해당 문자열 출현 빈도
분자 : 병합 후 subword xy의 빈도, 분모 : 각각의 개별 빈도 곱
score 값이 클수록 "x와 y를 합치는 것이 정보량 대비 효율적"임을 의미

토크나이저스

Tokenizers 라이브러리로 WordPiece 토크나이저를 구축·운영하는 과정

토크나이저 객체 생성

```
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
tokenizer = Tokenizer(WordPiece())
```

`WordPiece()` 모델 객체를 넘겨
내부에 WordPiece 알고리즘을 사용하도록 설정

정규화(Normalization) 및 사전 토큰화(Pre-tokenization) 설정

- 정규화
  텍스트를 NFD 유니코드 분해 형태로 바꾸고, 모두 소문자로 변경

```
from tokenizers.normalizers import Sequence, NFD, Lowercase
tokenizer.normalizer = Sequence([NFD(), Lowercase()])
```

- 사전 토큰화
  공백 및 구두점을 기준으로 문장을 미리 분할

```
from tokenizers.pre_tokenizers import Whitespace
tokenizer.pre_tokenizer = Whitespace()
```

말뭉치로 학습·저장

```
# train: corpus.txt 경로를 리스트로 전달
tokenizer.train(["../datasets/corpus.txt"])

# 훈련된 모델과 메타데이터를 JSON 형태로 저장
tokenizer.save("../models/petition_wordpiece.json")
```

로드·인코딩·디코딩

```
from tokenizers import Tokenizer
from tokenizers.decoders import WordPiece as WordPieceDecoder

# 저장된 JSON에서 토크나이저 복원
tokenizer = Tokenizer.from_file("../models/petition_wordpiece.json")
tokenizer.decoder = WordPieceDecoder()

# 단일 문장 또는 배치 인코딩
enc = tokenizer.encode("안녕하세요, 토크나이저가 잘 학습되었군요!")
batch = tokenizer.encode_batch([
    "이렇게 입력값을 리스트로 받아서",
    "쉽게 토크나이저를 사용할 수 있습니다"
])

# 토큰 목록과 정수 ID 확인
print(enc.tokens)   # → ['안녕하세요','.', '_토크','##나이저', …]
print(enc.ids)      # → [8760, 13, 10398, 7638, …]

# 정수 시퀀스를 다시 원문으로 복원
print(tokenizer.decode(enc.ids))
# → "안녕하세요. 토크나이저가 잘 학습되었군요!"
```

파이프라인

텍스트 정규화
→ 사전 토큰 분리
→ BPE/WordPiece 학습
→ JSON 저장 ↔ 로드
→ 토큰화·ID 인코딩 ↔ 역디코딩

06. 임베딩

텍스트 벡터화(Text Vectorization)

: 토큰화된 텍스트를 컴퓨터가 다룰 수 있는 숫자 벡터로 변환하는 과정
모델이 텍스트 간 유사도·관계를 수치적으로 계산할 수 있게 함

텍스트 벡터화 기초 기법

원-핫 인코딩(One-Hot Encoding)

- 어휘집 크기만큼의 차원을 갖는 벡터에서,
  해당 단어 색인 위치만 1, 나머지 0
- ex. `{I, like, apples, bananas}` →
  "I like apples" → `[1,1,1,0]`
  "I like bananas" → `[1,1,0,1]`
- 단점: 희소(sparse), 고차원, 의미 정보 미반영

빈도 벡터화(Count Vectorization)

- 문서 내 각 단어의 출현 횟수로 벡터값 설정
- ex. "apples"가 4번 등장하면 해당 차원 값은 4
- 단점: 차원·희소성 문제, 의미적 유사도 반영 안 됨

희소 벡터의 한계

- 고차원·희소(Sparse)
  어휘집 크기만큼의 차원을 갖지만, 실제 문장엔 극소수 토큰만 등장
  메모리·연산 비용 급증, "차원의 저주" 초래
- 의미 정보 부족
  원-핫/빈도 벡터는 단순 출현만 반영 → 문맥·의미 반영 불가
  "i scream for ice cream" vs. "ice cream for i scream" 모두 동일 벡터

→ 희소성·고차원 문제를 해결하고,
단어의 의미 관계를 벡터화하기 위해 **워드 임베딩** 사용

워드 임베딩(Word Embedding)

: 단어를 고정 길이의 실수 벡터로 표현해, 벡터 공간에서 의미적 관계를 학습

언어 모델

주어진 텍스트(문장)의 출현 확률을 추정하는 통계·딥러닝 모델
다음에 어떤 단어·문장이 올지를 확률로 계산해, 자연어 생성·이해의 기초로 활용

문장 확률 계산의 어려움

전체 문장 단위로 문장 확률을 직접 계산하려면

- 가능한 문장 조합이 사실상 무한함

- 비지도 학습으로 스스로 모든 조합의 확률을 학습해야 함

→ 완성된 문장 전체 확률을 한번에 구하는 것은 불가능에 가까움

이를 해결하기 위해, **자기회귀 언어 모델** 고안
자기 회귀 언어 모델은 문장 전체 대신
한 토큰씩 차례로 그 다음 토큰의 확률을 예측
→ 조건부 확률 예측을 반복해 문장을 생성·평가

자기회귀 언어 모델

주어진 토큰 시퀀스 $(w_1, \dots, w_{t-1})$를 조건으로
다음 토큰 w_t의 확률 $P(w_t|w_{1:t-1})$을 예측하는 모델

이전에 등장한 모든 토큰 정보를 활용해 문맥을 이해하고,
한 토큰씩 순차적으로 생성·예측

조건부 확률과 사슬 법칙

조건부 확률

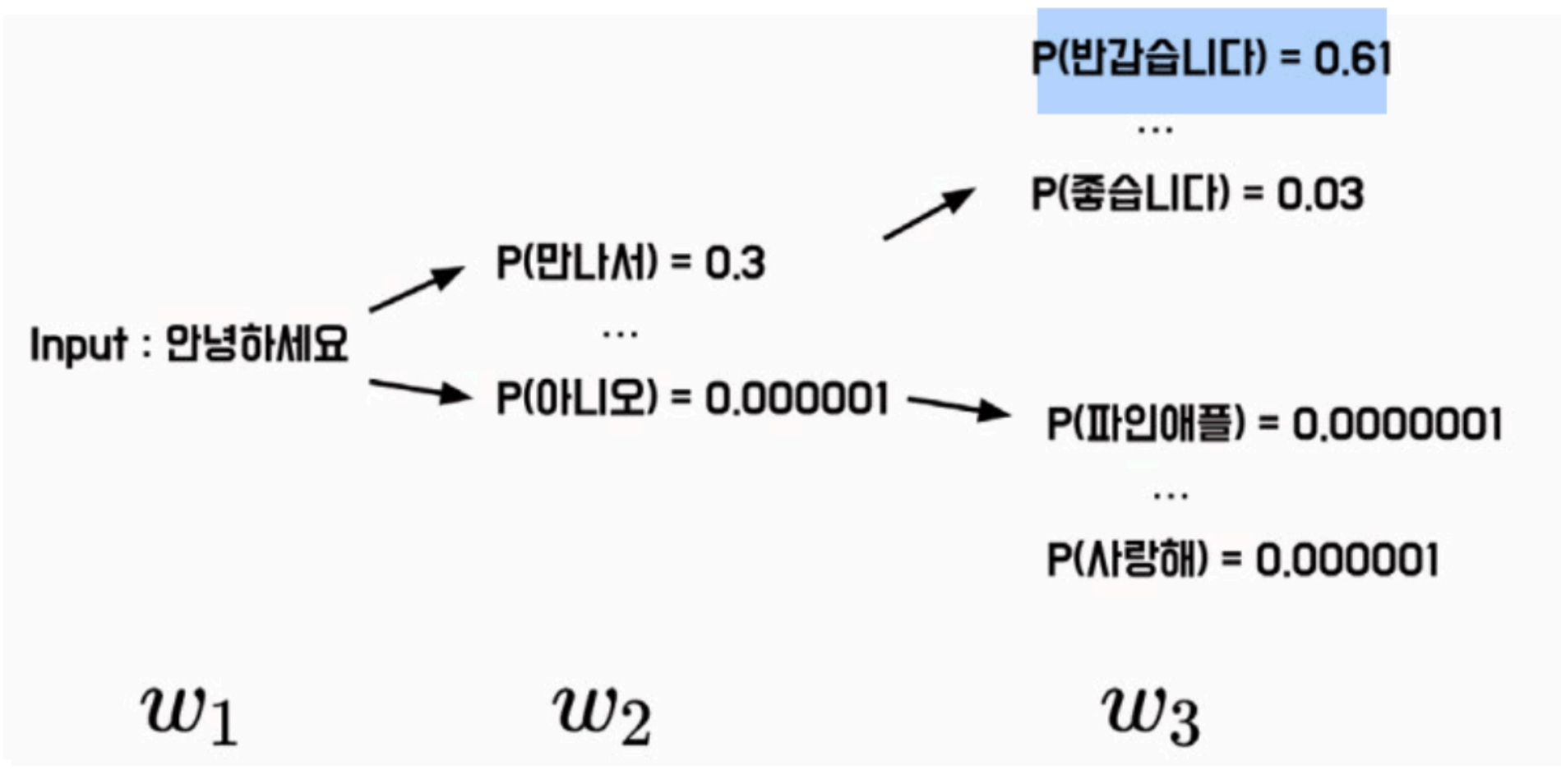
$$P(w_t|w_{1:t-1}) = \frac{P(w_{1:t})}{P(w_{1:t-1})}$$

사슬 법칙(chain rule) 적용

$$P(w_{1:n}) = \prod_{i=1}^{n} P(w_i|w_{1:i-1})$$

전체 문장 확률을 “첫 단어 확률” × “두 번째 단어의 조건부 확률” × ... 로 분해

자기회귀 언어 모델 구조



매 스텝마다 가장 높은 조건부 확률을 갖는 토큰을 선택하여 문장 생성

통계적 언어 모델 (Statistical LM)

말뭉치에 등장한 단어·문장 시퀀스의 빈도로부터 확률 분포를 추정해,
다음에 올 단어 또는 문장 전체의 확률을 계산하는 기법

마르코프 체인을 기반으로,
직전 n개의 토큰(또는 단어)의 빈도만 보고 다음 상태(토큰)를 예측

빈도 기반 조건부 확률

$$P(w_t|w_{1:t-1}) = \frac{\#(w_{1:t})}{\#(w_{1:t-1})}$$

ex. “안녕하세요”가 1,000번 등장했고, 그 뒤 “만나서”가 700번 이어졌다면,

$$P(\text{만나서}|\text{안녕하세요}) = \frac{700}{1000} = 0.7$$

사슬 법칙(chain rule) : $P(w_{1:n}) = \prod_{i=1}^{n} P(w_i|w_{1:i-1})$

장·단점 및 한계

장점

- 구현·추론이 단순하고 빠르며, 대규모 말뭉치에도 확장 가능

단점

- 데이터 희소성(Data sparsity)
  관측되지 않은 단어 조합은 확률 계산 불가(제로 확률)
- 맥락 부족
  n-그램 크기 이상 문맥 정보를 반영 못해, 긴 거리 의존 관계 무시
- OOV 문제: 사전에 없는 단어나 신조어에 대처 불가

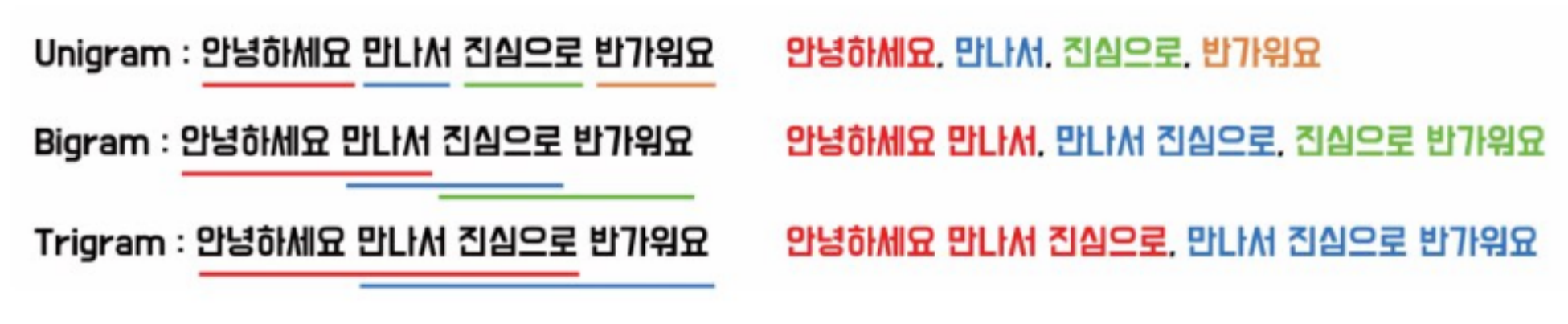
N-gram

텍스트에서 연속된 N개의 토큰(단어·글자 등)을 하나의 단위로 보고,
이 N-토큰 시퀀스가 나올 확률을 추정하는 통계적 언어 모델

N 값에 따른 명칭

- Unigram(1-gram): 개별 토큰
- Bigram(2-gram): 연속된 2토큰 묶음
- Trigram(3-gram): 연속된 3토큰 묶음
- N>3: 일반적으로 “N-gram”이라 부름

ex. 입력 문장을 유니그램 , 바이그램 , 트라이그램으로 분할


Unigram : 안녕하세요 만나서 진심으로 반가워요
안녕하세요. 만나서. 진심으로. 반가워요
Bigram : 안녕하세요 만나서 진심으로 반가워요
안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요
Trigram : 안녕하세요 만나서 진심으로 반가워요
안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

조건부 확률 계산

일반적인 언어 모델 :

$$P(w_{1:n}) = \prod_{i=1}^{n} P(w_i|w_{1:i-1})$$

N-gram 근사 (마르코프 가정) :

$$P(w_i|w_{1:i-1}) \approx P(w_i|w_{i-N+1:i-1})$$

→ “직전 N-1 토큰”만 보고 다음 토큰 확률 예측

장점 & 활용

장점

- 구현·추론이 매우 간단
- 작은 말뭉치에서도 의미 있는 연속 패턴(관용구, 콜로케이션) 추출 가능

활용 예시

- 관용표현 분석: “입이 무겁다” 같은 구(phrase) 추출
- 언어 모델링 기본 베이스라인
- 텍스트 분류·특징 추출: n-gram 피처로 사용

TF-IDF

BoW(Bag-of-Words)

문서·문장을 단어의 집합으로 보고, 등장 빈도만 세어 벡터로 표현

ex. [That movie is famous movie, I like this movie, I don't like this movie] 말뭉치를 BoW로 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

- 단어 순서·문맥 정보 무시
- 중복 출현 허용 → 단어별 TF 값(빈도) 사용

TF(Term Frequency)

문서 d 내에 단어 t가 등장한 횟수

$$\mathrm{TF}(t,d) = \mathrm{count}(t,d)$$

- 문서 길이가 길어질수록 TF 값도 커짐
- 자주 쓰이는 전문용어·관용어는 높은 TF, 일반 단어도 길면 TF가 높아짐

IDF(Inverse Document Frequency)

전체 말뭉치 D에서, 특정 단어가 얼마나 희귀한지 측정하는 값

$$\mathrm{IDF}(t,D) = \log\left(\frac{|D|}{|\{d \in D : t \in d\}|}\right)$$

$\mid D \mid$: 전체 문서 수
$\mid d : t \in d \mid$: 단어 t를 포함한 문서 수

많이 등장하는 일반 단어(관사·전치사 등)는 IDF 작아짐

TF-IDF

문서 내 상대적 중요도 가중치를 부여하기 위해 TF와 IDF를 곱한 값.
BoW(Bag-of-Words)에 가중치를 부여하는 방법

$$\text{TF-IDF}(t,d,D) = \mathrm{TF}(t,d) \times \mathrm{IDF}(t,D)$$

장점:

- 문서마다 “그 문서에 특화된 중요 단어”에 높은 가중치 부여

단점:

- 순서·문맥 무시
- 매우 짧은 문서나 소량 말뭉치에선 안정적이지 않을 수 있음

단어 빈도

한 문서(document) 내에서 특정 단어(term)가 등장한 횟수

$$\mathrm{TF}(t,d) = \mathrm{count}(t,d)$$

t : 단어(term)
d : 문서(document)
count(t,d) : 문서 d에서 단어 t가 등장한 횟수

문서가 길어질수록 단순히 많이 등장한 단어의 TF도 과대평가 될 수 있음

문서 빈도

특정 단어(term)가 몇 개의 문서(document)에 등장했는지 세는 값
→ 전체 문서 집합 D에서, 단어 t가 포함된 문서의 개수

$$\mathrm{DF}(t,D) = |\{d \in D : t \in d\}|$$

t : 단어(term)
D : 문서 집합(corpus)
$\{d \in D \mid t \in d\}$: 단어 t를 포함한 문서들의 집합
$\mid \cdot \mid$: 집합의 크기(문서 수)

DF가 높으면 거의 모든 문서에 등장
→ 너무 일반적(“the”, “is” 등) → 중요도 낮음

역문서 빈도

전체 문서 수를 특정 단어가 등장한 문서 수(DF)로 나눈 뒤 로그를 취한 값.
어떤 단어가 문서 집합에서 얼마나 희귀한지(중요한지)를 나타냄

$$\mathrm{IDF}(t,D) = \log\left(\frac{|D|}{1 + \mathrm{DF}(t,D)}\right)$$

$\mid D \mid$: 전체 문서 수
$DF(t,D)$: 단어 t가 등장한 문서 수

분모에 +1을 하는 이유
: DF가 0인(단어가 한 번도 등장하지 않은) 경우 분모가 0이 되는 것을 방지

DF 높음(흔한 단어) → 분모 커짐 → IDF 작아짐 → 중요도 ↓

TF-IDF

: 문서 내 단어 빈도(TF) × 전체 문서 대비 단어 희귀도(IDF)

TF–IDF 값이 크면
해당 문서에서 자주 등장하고, 전체 코퍼스에서는 드물게 등장하는
→ 핵심 단어로 간주

Scikit-learn 활용

```
from sklearn.feature_extraction.text import TfidfVectorizer

# 1) 벡터라이저 생성: 주요 매개변수
tfidf = TfidfVectorizer(
    input='content',        # 'content'(문자열) 또는 'filename'(파일 경로)
    encoding='utf-8',        # 인코딩
    lowercase=True,          # 모두 소문자 변환
    stop_words=None,         # 불용어 리스트
    ngram_range=(1,1),       # n-gram 범위: (최소 n, 최대 n)
    max_df=1.0,              # (비율 또는 정수) 이보다 자주 등장하면 불용어 처리
    min_df=1,                # (비율 또는 정수) 이보다 적게 등장하면 불용어 처리
    vocabulary=None,         # 사전(dict) 직접 지정 시 사용
    smooth_idf=True          # IDF 계산 시 분모에 +1 추가 여부
)

# 2) 학습·변환
corpus = [
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf.fit(corpus)               # 단어와 IDF 통계 학습
X = tfidf.transform(corpus)     # TF–IDF 행렬 생성
```

fit( ) : 코퍼스 전체를 읽고 → 단어 집합(vocabulary)과 IDF 계산
transform( ) : 각 문서를 TF–IDF 벡터로 변환
fit_transform( ) : 학습+변환 동시 수행

장점

- 문서 내/외 빈도 정보를 동시에 반영
- 간단히 벡터화+가중치 부여 가능
- 핵심 단어 추출, 문서 분류, 검색 등에 효과적

단점

- 순서·문맥 정보 손실 → “문장 생성”에는 부적합
- 단어 간 의미 관계(의미 유사도) 반영 못함

chelseey

이전 포스트
[Week 8]

다음 포스트
Ensemble Methods I

0개의 댓글

댓글을 작성하세요

댓글 작성

Powered by Stellate